# 04 · Context: Hidden States, the LLM's "Private Thoughts"

## The short version

As an LLM processes text, it builds one internal vector per token. These are its **hidden states**. They are its working understanding of the text so far, and in a chat they reflect the whole conversation. Normally they are used to pick the next word and then thrown away.

## Step 1: What an LLM does at every step

An LLM writes one token at a time. For each new token it runs the whole network, and the network's last layer produces a vector $\mathbf{h}_i$ (4,096 numbers for Qwen3-8B). A final small layer turns $\mathbf{h}_i$ into a probability for every possible next token, and one token is picked.

So for a query of $n$ tokens there are $n$ hidden states:

$$\mathbf{H} = [\mathbf{h}_1; \mathbf{h}_2; \ldots; \mathbf{h}_n] \quad (\text{an } n \times 4096 \text{ grid of numbers})$$

## Step 2: Hidden states carry context

Because each hidden state is computed while the model can see everything before it (the system prompt, the entire conversation, the query tokens written so far), $\mathbf{h}_i$ is not just about token $i$. It is "token $i$ *as understood in this conversation*".

When the LLM writes "Eiffel Tower height", the hidden state at

"height" already reflects that the user said "How tall is **it**?" and that "it" meant the tower. The text "Eiffel Tower height" keeps only a summary of that.

## Step 3: They are already paid for

Computing hidden states is the expensive part of running an LLM, and it has to be done anyway to generate the query. Keeping them costs memory but no extra computation. This is the crux of the paper: **the information is already sitting there**.

## Step 4: But they are in the "wrong language"

Hidden states can't be dropped into the index directly, for three reasons:

| Mismatch | Detail |
|---|---|
| **Wrong size** | 4,096 numbers per token, but the index uses 1,024 numbers per text. |
| **Wrong count** | One vector *per token*, but search needs one vector *per query*. |
| **Wrong space** | Hidden states are shaped to predict the next word. Embeddings are shaped so that similar meanings sit close together. The coordinates mean different things. |

There is also a subtler mismatch. LLMs are **causal**: token $i$ can only look at tokens before it, never after. Embedding models are usually **bidirectional**. So the first token's hidden state knows nothing about the rest of the query, while an embedding model sees the whole query at once. The paper points to this as a likely reason some queries are hard to translate (see file 18).

# Step 5: The translation task

So the job is clear: build a **translator** from "LLM hidden-state language" into "embedding-space language". It must fix the size, the count, and the space. That translator is the projection head described in files `10`–`16`.

---

**Key terms:** hidden state $\mathbf{h}_i$, last layer, causal vs. bidirectional, $d_h = 4096$, $d = 1024$.